

Tháng 3 2026

SINGLETON

Trình bày bởi Nhóm 5

NỘI DUNG

01. Vấn đề cần giải quyết

02. Định nghĩa singleton

03. Ý tưởng cốt lõi

04. Thành phần chính

05. UML

06. Luồng hoạt động

07. Đánh giá

08. So sánh

VẤN ĐỀ CẦN GIẢI QUYẾT

Đặt tình huống thực tế:

- Trong một số hệ thống, có những class chỉ nên có một object duy nhất trong toàn bộ chương trình.

Ví dụ:

- ConfigurationManager: quản lý cấu hình ứng dụng.
- Logger: ghi log tập trung.
- DatabaseConnectionManager: quản lý kết nối database.
- CacheManager: quản lý cache toàn cục.
- AppSettings: chứa thông tin setting dùng chung.
- FileManager: quản lý truy cập vào một file dùng chung.

ĐỊNH NGHĨA SINGLETON PATTERN

Singleton Pattern là một design pattern thuộc nhóm Creational Pattern.

Nó đảm bảo rằng một class chỉ có một instance duy nhất trong toàn bộ chương trình, đồng thời cung cấp một cách truy cập chung đến instance đó.

Nói đơn giản:

- Singleton là pattern dùng để tạo một object duy nhất và cho phép toàn hệ thống dùng chung object đó.

Ý TƯỞNG CỐT LÕI

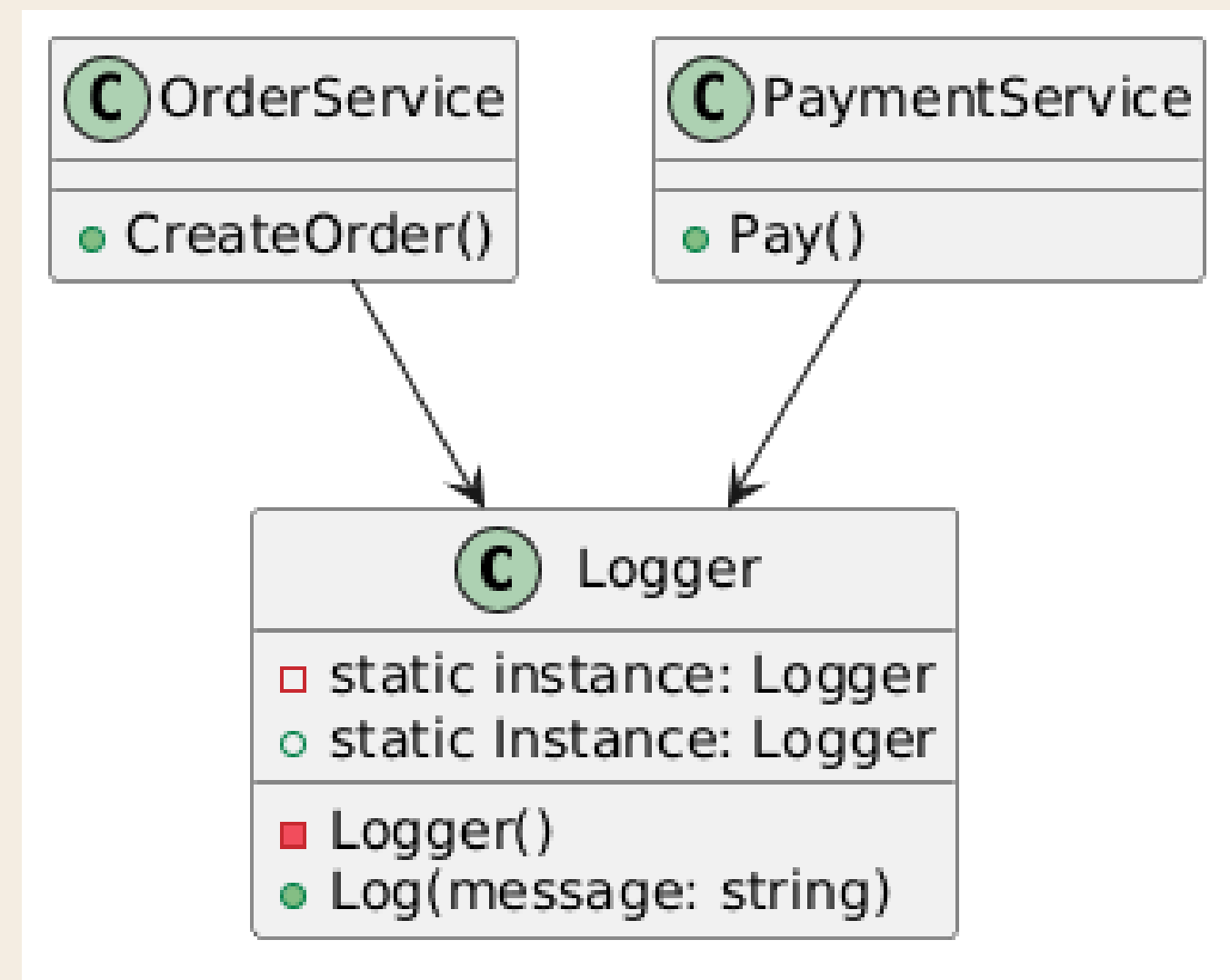
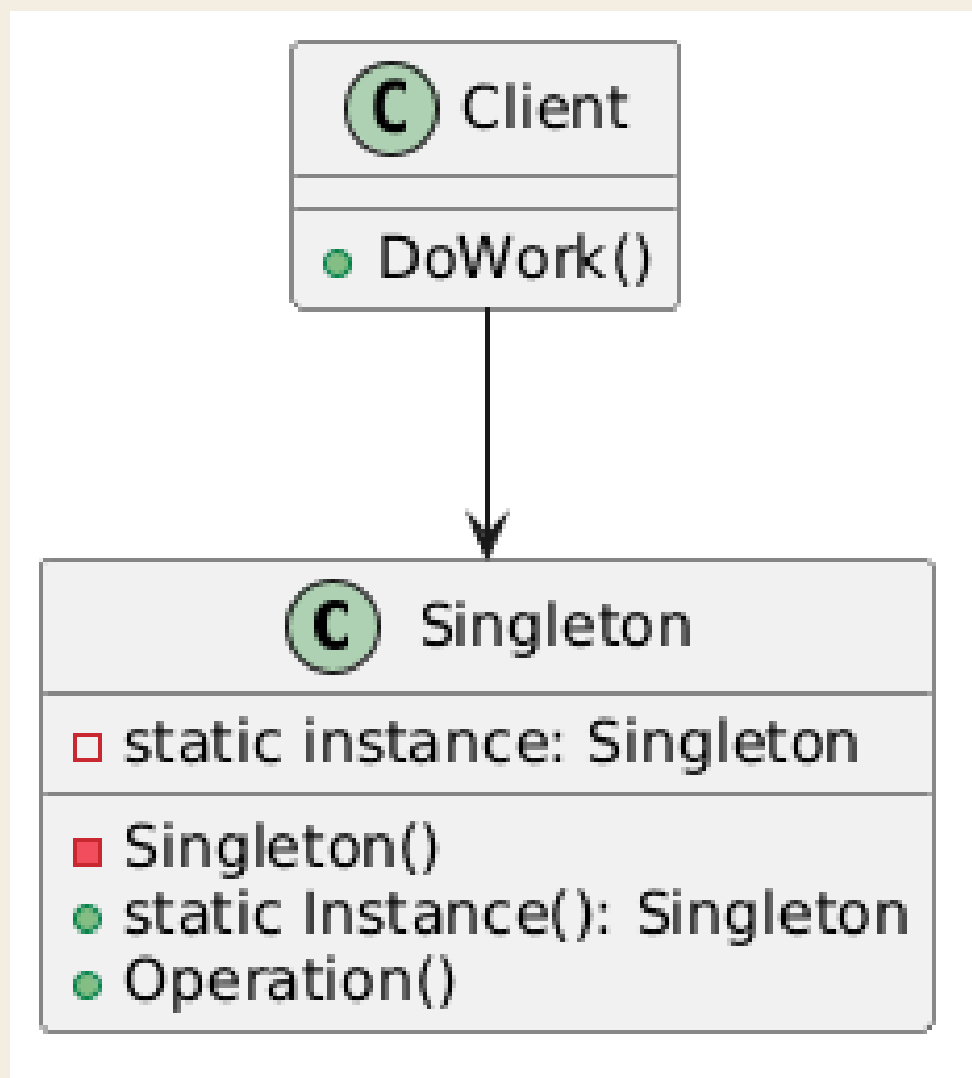
Ý tưởng chính của Singleton Pattern là:

- Ngăn client tự tạo object mới, rồi cung cấp một instance duy nhất được quản lý bên trong class.

THÀNH PHẦN CHÍNH

Thành phần	Vai trò
Singleton Class	Class tự quản lý instance duy nhất của chính nó
Private Constructor	Ngăn client tạo object mới bằng new
Static Instance	Lưu instance duy nhất
Static Accessor	Cung cấp điểm truy cập toàn cục
Client	Sử dụng instance thông qua accessor

UML



LUỒNG HOẠT ĐỘNG

Luồng hoạt động của Singleton Pattern:

1. Client cần dùng object singleton.
2. Client gọi Singleton.Instance.
3. Singleton kiểm tra instance đã được tạo chưa.
4. Nếu chưa có, Singleton tạo instance mới.
5. Nếu đã có, Singleton trả về instance cũ.
6. Client dùng instance đó để gọi method cần thiết.
7. Những lần gọi sau đều trả về cùng một instance.

ĐÁNH GIÁ

Ưu điểm	Nhược điểm
Đảm bảo chỉ có một instance	Dễ biến thành global state
Cung cấp điểm truy cập toàn cục	Làm tăng coupling
Có thể lazy initialization	Khó test
Có thể kiểm soát vòng đời object	Cần cẩn thận với đa luồng
	Có thể vi phạm Single Responsibility Principle

SO SÁNH

Singleton vs Static class

Tiêu chí	Singleton	Static Class
Bản chất	Một object duy nhất	Tập hợp member static
Có instance không?	Có	Không theo nghĩa object thông thường
Có implement interface được không?	Có	Không
Có kế thừa/đa hình không?	Có thể hạn chế nhưng vẫn linh hoạt hơn	Không
Có thể inject qua DI không?	Có, nếu đăng ký instance	Không phù hợp
Phù hợp khi	Cần một object duy nhất có hành vi và state	Hàm tiện ích không cần state

SO SÁNH

Singleton vs DI

Tiêu chí	Singleton Pattern truyền thống	Singleton lifetime trong DI
Ai quản lý instance?	Chính class đó	DI container
Client truy cập bằng	ClassName.Instance	Constructor injection
Coupling	Cao hơn	Thấp hơn
Testability	Khó test hơn	Dễ mock hơn
Phù hợp hiện đại?	Ít được khuyến khích hơn	Thường được ưu tiên hơn

SO SÁNH

Singleton vs Prototype

Tiêu chí	Singleton	Prototype
Nhóm pattern	Creational	Creational
Mục đích chính	Đảm bảo chỉ có một instance	Tạo object mới bằng cách clone object mẫu
Số lượng object	Một instance duy nhất	Nhiều object mới
Cách tạo	Lấy instance có sẵn	Clone từ prototype
Ví dụ	Một ConfigurationManager	Clone nhiều Enemy từ enemy mẫu

SO SÁNH

Singleton vs Flyweight

Tiêu chí	Singleton	Flyweight
Mục đích chính	Chỉ có một instance của một class	Chia sẻ nhiều object dùng chung theo trạng thái
Số lượng instance	Một	Có thể nhiều, mỗi object ứng với một intrinsic state
Trọng tâm	Kiểm soát instance	Tiết kiệm bộ nhớ
Ví dụ	Một Logger	Nhiều TreeType: Oak, Pine, Cherry

SO SÁNH

Singleton vs Factory

Tiêu chí	Singleton	Factory
Mục đích chính	Đảm bảo một instance duy nhất	Tạo object mà không để client biết chi tiết khởi tạo
Số lượng object	Một	Có thể nhiều
Client nhận object bằng	Instance	Create()
Trọng tâm	Quản lý vòng đời instance	Đóng gói logic tạo object
Ví dụ	Logger.Instance	PaymentFactory.Create("momo")

Tháng 3 2026

THANK YOU

for Listening

Trình bày bởi Nhóm 5